Cascading Style Sheet

Gnanavel Durairaj

Content

On Day - 1

1. What is CSS and Why is it Used
2. Inline, Internal, and External CSS
3. Basic CSS Syntax: Selectors and Declarations
4. Grouping and Combinator Selectors
5. Pseudo-classes
6. Pseudo-elements
7. CSS Box Model
8. Box-Sizing: content-box vs border-box
9. Width, Height, and Overflow Properties
10. Exercise & Quiz

What is CSS and Why is it Used?

- **CSS (Cascading Style Sheets)** is a language used to style and layout web pages. It controls how HTML elements look on the screen, including colors, fonts, spacing, and positioning.
- **Why use CSS?**
  - Separates content (HTML) from presentation (styling).
  - Makes websites visually appealing.
  - Enables consistent styling across multiple pages.
  - Simplifies responsive design for different devices.

Inline, Internal, and External CSS

- **Inline CSS**
  - CSS is written directly inside an HTML element using the style attribute.
  - **Use Case**: Quick styling for a single element. Not recommended for large projects.

```
<p style="color: red; font-size: 20px;">This is a red paragraph.</p>
```

Example for Inline CSS

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Inline CSS Example</title>
</head>
<body>
    <h1 style="color: blue; text-align: center;">Welcome to My Website</h1>
    <p style="font-size: 18px; color: green;">This is a paragraph styled with inline CSS.</p>
    <button style="background-color: orange; color: white; padding: 10px 20px;">Click Me</button>
</body>
</html>
```

In this example, the `style` attribute is used within the HTML elements like
`<h1>`, `<p>`, and `<button>`
to apply CSS styles directly to them.

Inline, Internal, and External CSS

- **Internal CSS**
  - CSS is written inside a <style> tag in the <head> section of an HTML file.
  - **Use Case:** Useful for small projects or single-page styling.

```
<head>
  <style>
    p {
      color: blue;
      font-size: 18px;
    }
  </style>
</head>
<body>
  <p>This is a blue
paragraph.</p>
</body>
```

Example for Internal CSS

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Internal CSS Example</title>
    <style>
        h1 {color: blue; text-align: center;}
        p {font-size: 18px;color: green;}
        button {background-color: orange;
            color: white;  padding: 10px 20px;  border: none;
            cursor: pointer; }
         button:hover { background-color: darkorange;  }
    </style>
</head>
<body>
    <h1>Welcome to My Website</h1>
    <p>This is a paragraph styled with internal CSS.</p>
    <button>Click Me</button>
</body>
</html>
```

In this example, the CSS is placed inside the `<style>` tag within the `<head>` section of the document. This approach is ideal for styling the entire page without repeating styles across different HTML elements.

Inline, Internal, and External CSS

- **External CSS**
  - CSS is written in a separate .css file and linked to the HTML file using the <link> tag.
  - **Use Case:** This method is great for maintaining a clean, efficient codebase and for applying consistent styles across multiple pages on a website.

Example for External CSS

Index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>External CSS Example</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This is a paragraph styled with external CSS.</p>
  <button>Click Me</button>
</body>
</html>
```

Style.css

```css
h1 {color: blue;
   text-align: center;}

p {font-size: 18px;
   color: green;}

button {
   background-color: orange;
   color: white;
   padding: 10px 20px;
   border: none;
   cursor: pointer;}

button:hover {
   background-color:
darkorange;
}
```

Basic CSS Syntax: Selectors and Declarations

❖ **CSS Syntax consists of:**

- **Selector:** Targets the HTML element(s) to style.
- **Declaration Block:** Contains one or more declarations in curly braces {}.
- **Declaration:** A property-value pair that defines the style.

```
p {
  color: purple; /* Property: color, Value: purple */
  font-size: 20px; /* Property: font-size, Value: 20px
*/
}
```

- **Explanation:**
  - p is the **selector** (targets all <p> elements).
  - color: purple; is a **declaration** where color is the property and purple is the value.
  - Multiple declarations are separated by semicolons ;

Basic CSS Syntax: Selectors and Declarations

❖ **Common Selectors:**

1. **Element Selector**:
   - Targets all <h1> elements.
2. **Class Selector:**
   - Targets elements with class="highlight“
3. **ID Selector:**
   - Targets the element with id="header".

```
h1 {  color: red;}
```

```
.highlight {  background:yellow;}
```

```
#header {  font-size: 24px;}
```

```
<body>
  <h1 id="header">Css Day 1</h1>
    <h1>Welcome to My Website</h1>
    <p class="highlight">This is a paragraph  with  CSS.</p>

</body>
```

Grouping Selectors

1. Grouping Selectors

- Applies the same styles to multiple selectors.
- Separate selectors with a comma (,).
- This styles all <h1>, <h2>, and <h3> elements with green text.

```
h1, p, button {
    font-family: Arial, sans-serif;
    font-size: 18px;
    color: darkblue;}
h1 { text-align: center;
    font-size: 30px;}
button {    background-color: orange;
    color: white;
    padding: 10px 20px;
    border: none;
    cursor: pointer;}
button:hover {
    background-color: darkorange;}
```

Combinator Selectors

Combine selectors to target specific relationships between elements.

- Types:
  1. Descendant Selector ( ):
     - Targets elements inside another element.
  2. Child Selector(>):
     - Targets direct children of an element.
     - This styles only <li> elements that are direct children of <ul>.
  3. Adjacent Sibling Selector(+)
     - Targets the element immediately after another element.
     - This styles the <p> element that comes right after an <h1>.
  4. General Sibling Selector(~)
     - Targets all siblings after an element.
     - This styles all <p> elements that are siblings of an <h1>.

```
div p {  color: red;}
```

```
ul > li {  list-style:square;}
```

```
h1 + p {font-weight:bold;}
```

```
h1 ~ p {color:purple;}
```

Example for Combinator Selectors

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Combinator Selectors Example</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <div class="container">
        <h1>Welcome to My Website</h1>
        <p>This is a paragraph inside the container.</p>
        <div>
            <p>This paragraph is nested within another div.</p>
        </div>
    </div>
    <h2>Adjacent Sibling Example</h2>
    <p>This paragraph will be styled differently due to the adjacent sibling selector.</p>
    <h2>General Sibling Example</h2>
    <p>This paragraph will also be styled differently due to the general sibling selector.</p>
</body>
</html>
```

Example for Combinator Selectors

CSS (styles.css):

```
/* Descendant selector: Selects all p elements inside .container */
.container p {
    color: green;
    font-size: 18px;
}
/* Child selector: Selects only direct child divs of .container */
.container > div {
    background-color: lightblue;
    padding: 10px;
}
/* Adjacent sibling selector: Selects the first p immediately after h2 */
h2 + p {
    color: orange;
    font-weight: bold;
}
/* General sibling selector: Selects all p elements that follow an h2 */
h2 ~ p {
    color: purple;
}
```

Combinator Selectors

1. Descendant Selector (`.container p`):
   - It selects all `<p>` elements that are inside any element with the class `.container`. This affects both `<p>` elements in the HTML.

2. Child Selector (`.container > div`):
   - It selects any `<div>` that is a direct child of an element with the class `.container`. This only affects the immediate `<div>` within `.container`.

3. Adjacent Sibling Selector (`h2 + p`):
   - It selects the first `<p>` element immediately following an `<h2>` element. In the HTML, this will style the first `<p>` after `<h2>`, which is orange and bold.

4. General Sibling Selector (`h2 ~ p`):
   - It selects all `<p>` elements that are siblings (i.e., are at the same level in the DOM) and follow an `<h2>`. This affects the second `<p>` under the "General Sibling Example" section, making it purple.

This allows for precise control over the styling of elements based on their structural relationships.

Pseudo-classes

Common Pseudo-classes:

1. **:hover:**
   - Styles an element when the mouse hovers over it.
   - Changes link color to red on hover.

```
a:hover { color: red;}
```

2. **:focus:**
   - Styles an element when it is focused (e.g., input field).
   - Changes border color of an input field when focused.

```
input:focus {border-color: blue;}
```

3. **:nth-child():**
   - Targets elements based on their position in a parent.
   - Styles every odd <li> element with a light gray background.

```
li:nth-child(odd) {  background: lightgray;}
```

Pseudo-classes

Common Pseudo-classes:

4. :first-child:

```
li:first-child {  background: lightblue;}
```

- Targets first elements based on their position in a parent.
- Styles first <li> element with a light blue background.

```
li:last-child {  background: lightgreen;}
```

5. :last-child:

- Targets first elements based on their position in a parent.
- Styles first <li> element with a light green background

6. :active

```
button:active
{  background-color: darkorange;
   color: white;}
```

- Targets elements based on the event
- Styles button on click of the event.

Pseudo-classes Example

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Pseudo-class Example</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <h1>Welcome to My Website</h1>
    <p>This is a normal paragraph.</p>
    <p>This is another paragraph. Hover over me!</p>
    <p>This is the last paragraph in the list.</p>
    <button>Click Me</button>
    <input type="text" placeholder="Focus on me!">
</body>
</html>
```

Pseudo-classes Example

```
p:hover {color: red;
    font-weight: bold;}

p:first-child {font-style: italic;
    color: blue;}

p:last-child {  color: green;}

input:focus {
    border-color: orange;
    background-color: yellow;
}

button:active {
    background-color: darkorange;
    color: white;
}
```

1. `:hover`:
   - When the user hovers over the second `<p>` (which is the one with the text "Hover over me!"), it will turn red and bold.
2. `:first-child`:
   - The first `<p>` element in the HTML will have an italicized blue font.
3. `:last-child`:
   - The last `<p>` element in the HTML will be styled with a green color.
4. `:focus`:
   - When the input field gains focus (such as when the user clicks on it or tabs into it), its border turns orange and the background turns yellow.
5. `:active`:
   - When the user clicks on the button, the background will change to dark orange, and the text will turn white.

These pseudo-classes allow you to enhance the interactivity and styling of elements in response to user actions or document structure!

Pseudo-elements

Pseudo-elements

- Targets specific parts of an element.
- Starts with double colons (::).

Common Pseudo-elements:

1. **::before:**
   - Inserts content before an element.
   - Adds "Note: " before every <p> element.

```
p::before {
  content: "Note: ";
  font-weight: bold;
}
```

2. **::after:**
   - Inserts content after an element.
   - Adds "(Read More)" after every <p> element.

```
p::after {
  content: " (Read More)";
  color: blue;
}
```

CSS Box Model

Understanding Margin, Padding, Border, and Content

The CSS Box Model describes how elements are structured as boxes with four layers:

Content:

- The innermost part of the box where text, images, or other content appears.
- Controlled by width and height.

Padding:

- The space between the content and the border.
- Adds space inside the element.
- Controlled by padding.

Border:

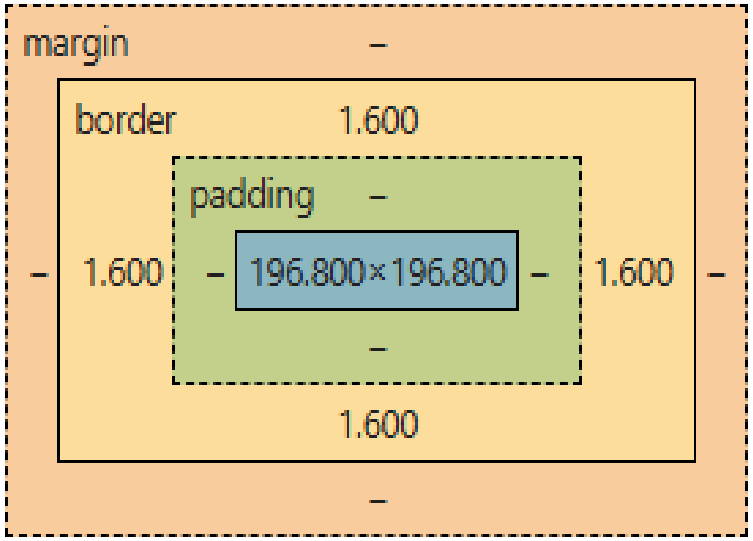
- The edge surrounding the padding and content.
- Controlled by border.

Margin:

- The space outside the border, separating the element from other elements.
- Adds space outside the element.
- Controlled by margin.

CSS Box Model

Visual Representation:



Example:

```
.box {
  width: 200px; /* Content width */
  height: 100px; /* Content height */
  padding: 20px; /* Space inside */
  border: 5px solid black; /* Border */
  margin: 10px; /* Space outside */
}
```

Total width = width + padding + border + margin = 200 + 20*2 + 5*2 + 10*2 = 270px.

Box-Sizing: content-box vs border-box

The box-sizing property controls how the total width and height of an element are calculated.

content-box (Default):

- Width and height only apply to the content.
- Padding and border are added outside the content

```
.box {
  box-sizing: content-box;
  width: 200px;
  padding: 20px;
  border: 5px solid black;
}
```

- Total width = 200 + 20*2 + 5*2 = 250px.

Box-Sizing: content-box vs border-box

border-box:

- Width and height include **content, padding, and border.**
- Padding and border are included inside the specified width and height.

```
.box {
  box-sizing: border-box;
  width: 200px;
  padding: 20px;
  border: 5px solid black;
}
```

- Total width = 200px (includes padding and border).

Width, Height, and Overflow Properties

Width and Height:

- Control the size of the content area.
- Can also use relative units like %, vw, vh, or em.

```
.box {
  width: 300px; /* Fixed width */
  height: 150px; /* Fixed height */
}
```

Width, Height, and Overflow Properties

Overflow:

Controls what happens when content overflows the box.

Values:

- **visible (Default):** Content overflows outside the box.
- **hidden:** Content is clipped and hidden.
- **scroll:** Adds scrollbars to view overflowed content.
- **auto:** Adds scrollbars only when needed

```
.box {
  width: 200px;
  height: 100px;
  overflow: auto; /* Adds scrollbars if content overflows */
}
```

Example with different Units

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Responsive Box Example</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <div class="box">
        This box uses different CSS units like %, vw, vh, and em.
    </div>
</body>
</html>
```

Example with different Units

```
.box {
    width: 50%;     /* 50% of the parent container's width */
    height: 30vh;  /* 30% of the viewport height */
    margin: 5% auto;     /* 5% margin from the top and bottom, auto for centering horizontally */
    padding: 2em;    /* Padding relative to the font size of the element */
    font-size: 1.5em;    /* 1.5 times the base font size */
    background-color: lightcoral;
    color: white;
    text-align: center;
    border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
}
```

Example with different Units

1. `width: 50%’ ;
   - The width of the `.box` is set to 50% of its parent element (the body in this case). It will adjust automatically as the size of the parent changes.

2. `height: 30vh`:
   - The height of the `.box` is set to 30% of the viewport's height (the height of the browser window). So as the window resizes, the height will scale accordingly.

3. `margin: 5% auto`:
   - The box has a margin of 5% on the top and bottom. The `auto` value horizontally centers the box within the parent container.

4. `padding: 2em`:
   - The padding is set relative to the font size of the `.box`. Since the font size is set to `1.5em`, the padding will be 2 times that base font size (roughly 3 times the default font size).

5. `font-size: 1.5em`:
   - The font size is set to 1.5 times the base font size of the page, making the text larger relative to the default size.

6. `background-color: lightcoral;`:
   - The background color is a light coral shade.

7. `border-radius: 10px;`:
   - The box has rounded corners with a radius of 10px.

8. `box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);`:
   - A subtle shadow is applied to the box for some depth.

Width, Height, and Overflow Properties

Overflow:

Controls what happens when content overflows the box.

Values:

- **visible (Default):** Content overflows outside the box.
- **hidden:** Content is clipped and hidden.
- **scroll:** Adds scrollbars to view overflowed content.
- **auto:** Adds scrollbars only when needed

```
.box {
  width: 200px;
  height: 100px;
  overflow: auto; /* Adds scrollbars if content overflows */
}
```

Exercise and Quiz

Thanks – End of Day 04

Edited by Gnanavel Durairaj

24-Feb-2025